

EDD (Earliest Deadline First, 最早截止优先)

定义

将所有包裹按 T_i (最晚送达时间) 从小到大排序, 依次决定装车。

为什么 EDD 对 T3 有用

T3 中 $S_i = 0$, 所有包裹立即可送, 唯一的时效约束是 $D_i \leq T_i$ 。

EDD 的核心性质: 在**单机调度**中, EDD 是最小化最大延迟 ($L_{\max}$) 的最优策略。类比到 T3——在同一条"车"这个资源上串行处理所有包裹时, **把 deadline 近的包裹优先送**, 能最大程度避免它们超时。

举例:

- 包裹 A: $T_A = 10$, 目的地远 (需 8 时间单位)
- 包裹 B: $T_B = 20$, 目的地近 (需 2 时间单位)

若先送 B 再送 A: B 在 2 时刻送达 (OK), A 在 10 时刻送达 (刚好不超时)

若先送 A 再送 B: A 在 8 时刻送达 (OK), B 在 10 时刻送达 (OK)

这个例子两者都不超时, 但如果 A 的 deadline 是 9, 先送 B 就会导致 A 超时——EDD 杜绝了这种情况。

在 T3 中的角色: 分趟策略

EDD 只决定**谁先进趟**, 不决定趟内顺序。流程:

1. `pkgs_sorted = sort(pkgs, key=deadline)` (EDD 排序)
2. 贪心装车: 遍历排序后的包裹, 当前趟装得下就装, 装不下就封趟开新趟
3. 每趟内部用 `heaviest-first` 排目的地 (降成本)

EDD 保证 deadline 紧的先出发, `heaviest-first` 保证重先卸降成本。两者分工明确。

评分函数的改进

你说的"超时包裹的成本 $\times 2$ "本质上是一种**个性化惩罚**——重包裹超时比轻包裹超时更严重 (因为重包裹本该贡献更多"有效成本")。这比统一 λ 更精细。

数学表达式

设包裹 i 的"有效成本贡献"为其在配送过程中实际产生的路段成本之和：

$$\text{pkg_cost}_i = \sum_{\text{seg}: i \in \text{remain}(\text{seg})} \text{dist}[\text{seg}] \times w_i$$

评分函数：

$$\text{score} = \text{total_cost} + \alpha \cdot \sum_{i: D_i > T_i} \text{pkg_cost}_i$$

取 $\alpha = 1$ 即"超时包裹的成本翻倍计入"。这个做法的好处是不需要手动调 λ ，惩罚量由包裹自身重量和路径自动决定。

实现上稍复杂但可行

需要在 `evaluate_solution` 中逐包裹追踪它的成本贡献。`trip_cost` 函数目前按段累计、不区分包裹。可以在模拟时额外维护 `pkg_cost[i]`：

```
// 逐段模拟时
for each segment:
    for each pkg still on car:
        pkg_cost[pkg] += seg_len * pkg.weight;
```

复杂度 $O(\text{趟数} \times \text{段数} \times \text{包裹数})$ ，和 T2 的 SA 同一量级，可接受。

要我切 ACT MODE 开始实现吗？先读完当前 `task3.h` 和 `task3.cpp` 的实际内容，然后按这个方案写代码。